

Candidate Thesis Brief

A point of view on modernizing manufacturing applications without losing production continuity or data meaning.

EXECUTIVE THESIS

Don't migrate screens. Prove the signal path.
Run legacy and target behavior side by side, reconcile data context, and cut over only when function, data, ownership, support, and rollback evidence are explicit.

THE WORK TO BE DONE

Modernize legacy MES functionality with Ignition while protecting production continuity; improve SQL-backed manufacturing applications; strengthen the AVEVA PI historian environment; and create dependable manufacturing data integration into a cloud data lake.

WHY THIS MATTERS NOW

Legacy behavior

Business-critical logic may be spread across screens, queries, scripts, workarounds, and local knowledge.

Historian entropy

Duplicate, stale, poorly named, or weakly contextualized signals increase support burden and reduce analytic trust.

Cloud demand

Raw plant signals are not usable data products until meaning, quality, ownership, and intended use travel with them.

Contract clock

The team needs visible progress through year-end without purchasing speed with maintainability or production risk.

CORE TENSIONS

Speed / continuity

Deliver useful application progress without treating production as a test environment.

Standards / local reality

Build reusable Ignition patterns without erasing legitimate plant differences.

Cleanup / dependency risk

Improve PI and SQL context without breaking unknown downstream consumers.

Access / ownership

Make plant data useful in the cloud while preserving OT reliability, security, and stewardship.

SIGNAL TWIN OPERATING MODEL

Stage	Primary question	Required evidence	Decision
Read	What does the live function actually do?	States, exceptions, users, SQL, tags, consumers, failure behavior	Bound scope
Twin	What is the smallest supportable replacement?	UDTs/tags, components, logic, interfaces, diagnostics, source control	Build / revise
Reconcile	Does target data carry stable meaning?	Naming, timestamps, units, quality, PI context, SQL schema, ownership	Accept / repair
Prove	Does the target behave under real conditions?	Parity, exceptions, performance, operator use, rollback	Promote / hold
Release	Can the team support and learn from it?	Runbook, owner, observation, incident path, reusable pattern	Cut over / return

Alignment and Contribution

Verified evidence selected for the work, followed by the contribution sequence and decision questions.

EVIDENCE ALIGNED TO THE WORK

Role need	Verified evidence	Transfer logic
Manufacturing environment	Vape-Jet software-enabled robotics; Compunetics advanced electronics; Amazon continuous operations	Frontline consequence, production continuity, quality, escalation, and live-system change.
MES and application modernization	ERP/Odoo and cross-workflow transformation; standard-work and process-control mechanisms	Map current work, define target state, create ownership, and install repeatable mechanisms.
SQL and data workflows	SQL toolkit; telemetry, dashboards, workflow architecture, analytics, and agentic-system design	Connect application behavior to schemas, operational context, evidence, and downstream decisions.
Industrial reliability	High-reliability electronics, quality systems, root cause, safety, and 24/7 operations	Consequence-aware testing, release, rollback, diagnostics, and support discipline.
Cross-functional delivery	Manufacturing, engineering, quality, operations, technology, support, customers, and leadership	Translate competing needs into explicit interfaces, priorities, decisions, and owners.

IMMEDIATE CONTRIBUTION

Read the live system Map behavior, dependencies, users, workarounds, data consumers, ownership, and failure consequence.	Build supportable change Make states, interfaces, diagnostics, testing, release criteria, and rollback explicit.	Make data usable Stabilize names, timestamps, units, quality, asset relationships, semantics, and stewardship.
--	--	--

QUESTIONS I WOULD BRING INTO THE ROOM

Which functions must be modernized by year-end, and why those first?	What is the current Ignition architecture, module mix, deployment pattern, and version?
Where is ownership ambiguous across manufacturing, controls, IT, data, R&D, and vendors?	Which PI or SQL changes require a consumer-impact map before action?
What must the cloud data lake enable that current plant systems cannot?	What would make a contract extension an obvious business decision?

SUCCESS CONDITIONS

Safer releases Validated behavior, explicit rollback, named authority, and observed post-release performance.	Trustworthy data Clear context, quality, stewardship, consumer mapping, and accepted cloud use.	Compounding capability Reusable components, tests, diagnostics, runbooks, and a prioritized next tranche.
---	---	---

DECISION PRINCIPLES

Behavior before resemblance Preserve what the live function must do, not merely how its screen looks.	Meaning before volume Make plant data trustworthy before sending more of it downstream.	Support before scale Leave diagnostics, ownership, and reusable patterns with every release.
---	---	--

PLATFORM CONTEXT

My verified record is strongest in industrial systems, manufacturing operations, SQL/data workflows, modernization mechanisms, reliability, and technical translation. It does not include documented years of production Ignition development or AVEVA PI administration; the value case rests on the broader system integration and operating discipline this work requires.

2 / 2